

خمس قيم لخاصية Hibernate ddl-auto

شرح تفصيلي لكل قيمة: وظيفتها، طريقة عملها، مثال عملي، واستخدامها الحقيقي في سوق العمل — مع جدول مقارنة نهائي.

hibernate.hbm2ddl.auto

spring.jpa.hibernate.ddl-auto

none

1

بدون أي تدخل في الـ Schema

1. بتعمل إيه؟

Hibernate ما بيلمسش الـ schema خالص — مش بيتحقق، ومش بيعمل create أو update لأي جدول.

2. إزاي بتشتغل؟

دي القيمة الافتراضية في Spring Boot لو مفيش embedded database. إدارة الـ schema بتبقى مسؤولية أداة تانية تمامًا زي Flyway أو Liquibase، أو DBA بيدير الجداول يدويًا.

3. مثال بسيط عليها

```
spring.jpa.hibernate.ddl-auto=none
# Entity: User(id, name)
# DB: USER table must already exist – Hibernate does not create or modify it
```

Case من سوق العمل — إمتى نستخدمها؟

في أي شركة بتدير الـ database schema رسميًا بأداة migration زي Flyway. الفريق بيكتب migration scripts مُراجعة (reviewed) بدل ما يسبب Hibernate يعدّل الجداول تلقائيًا — ده اللي بيحصل عمليًا في أغلب فرق الـ backend الاحترافية.

1. بتعمل إيه؟

بيقارن الـ entities بتاعتك مع الجداول الموجودة فعلياً في الداتابيز، ويتأكد إنهم متطابقين تماماً — من غير ما يعدل أي حاجة.

2. إزاي بتشتغل؟

وقت الـ startup، لو لقي أي اختلاف (عمود ناقص، نوع بيانات مختلف...) بيرمي exception ويوقف تشغيل التطبيق فوراً، عشان نكتشف المشكلة بدري.

3. مثال بسيط عليها

```
spring.jpa.hibernate.ddl-auto=validate
# Entity: Team(id, name, playerCount)
# DB: TEAM must contain matching columns → otherwise startup fails
```

Case من سوق العمل — إمتى نستخدمها؟

في بيئة الـ production، كخطوة أمان قبل أو أثناء الـ deployment، عشان نتأكد إن الكود الجديد متوافق فعلياً مع الـ schema الحالية قبل ما يوصل لليوزر — من غير أي خطر تعديل أو مسح بيانات حقيقية.

1. بتعمل إيه؟

بيقارن الـ entities مع الـ schema الموجود، وبيضيف اللي ناقص بس (جداول أو أعمدة جديدة) — من غير ما يمسح أي بيانات موجودة بالفعل.

2. إزاي بتشتغل؟

وقت الـ startup، بيعمل مقارنة metadata وبيطلق أوامر ALTER TABLE للفروقات الجديدة فقط. لاحظ إنه مش بيضيف الأعمدة اللي اتشالت من الـ entity، فمش أداة migration كاملة.

3. مثال بسيط عليها

```
spring.jpa.hibernate.ddl-auto=update
# Entity User كانت: id, name
# جديد field أضفت: phoneNumber
# → Hibernate يحاول إضافة PHONE_NUMBER
# البيانات القديمة تظل موجودة
```

Case من سوق العمل — إمتى نستخدمها؟

في مرحلة الـ development النشط، لما تكون بتضيف وتعديل fields باستمرار وعايز تحافظ على بيانات التجربة (test data) من غير ما تمسحها في كل تشغيل جديد للتطبيق.

1. بتعمل إيه؟

بيعمل **DROP** لكل الجداول الموجودة، ويعيد **CREATE** من جديد بالكامل حسب الـ **entities** الحالية.

2. إزاي بتشتغل؟

بيتنفذ مرة واحدة عند الـ **startup** بس. يعني كل مرة تشغل التطبيق، الجداول القديمة بتتمسح وتعمل جداول فاضية جديدة — أي بيانات سابقة بتضيع.

3. مثال بسيط عليها

```
spring.jpa.hibernate.ddl-auto=create
# عند تشغيل التطبيق:
# DROP TEAM → CREATE TEAM من جديد
# تختفي TEAM أي بيانات قديمة في →
```

Case من سوق العمل — إمتي نستخدمها؟

في بداية مشروع جديد، أو لما تعمل **demo** أو **proof-of-concept** سريع وعازل تتأكد إن الـ **schema** بتاعتك متطابقة مع الـ **entities** من غير ما يهيك مصير البيانات القديمة.

1. بتعمل إيه؟

زي **create** بالظبط (**DROP** ثم **CREATE** عند البداية)، لكن كمان بيعمل **DROP** إضافي للجداول لما التطبيق يقفل.

2. إزاي بتشتغل؟

CREATE عند الـ **startup**، ثم **DROP** عند الـ **shutdown**. الـ **schema** بتظهر وتختفي تمامًا مع دورة حياة التطبيق نفسه.

3. مثال بسيط عليها

```
spring.jpa.hibernate.ddl-auto=create-drop
# عند startup: DROP → CREATE
# أثناء الاختبار: استخدم الجداول
# عند shutdown: DROP
# يبدأ ببيئة نظيفة Test كل →
```

Case من سوق العمل — إمتي نستخدمها؟

في **unit / integration tests**، خصوصًا مع **H2 in-memory database** — كل تشغيل تست ياخذ بيئة نظيفة تمامًا من غير ما يسبب أي أثر بعد ما يخلص، وده معيار شائع جدًا في فرق الـ **QA** والـ **backend**.

★ جدول المقارنة النهائي

migration tool (Flyway/Liquibase) مع Production	—	× لا شيء	× لا	none
Production — فحص أمان قبل التشغيل	Startup	× (فحص فقط)	× لا	validate
Development النشط مع الحفاظ على بيانات التجربة	Startup	✓ إضافة فقط	× لا (إضافة فقط)	update
مشروع جديد / Demo / Proof-of-concept	Startup	✓ إعادة إنشاء كاملة	✓ نعم (Drop كامل)	create
Unit / Integration Testing	Startup + Shutdown	✓ إعادة إنشاء كاملة	✓ نعم (بداية ونهاية)	create-drop

قاعدة سريعة للحفاظ: مستخدمش **create**، **create-drop**، ولا حتى **update** في production أبدًا — دول لك development والـ testing بس. في production استخدم **validate** أو **none** مع أداة migration زي Flyway.